

Retail Sales Analysis Project

Simran Sandhu | SQL & Tableau Portfolio Project | January to March 2026

PROJECT OVERVIEW

Dataset	UCI Online Retail, 541,909 transactions, UK online retailer 2010 to 2011
Source	Kaggle / UCI Machine Learning Repository
Tools	SQL (SQLite), DB Browser for SQLite, Tableau Public
SQL Techniques	GROUP BY, Subqueries, CASE WHEN, CTEs, LAG, RANK, CTE JOIN, Data Validation, Window Functions
Dashboard	public.tableau.com/app/profile/simran.sandhu1920/viz/retailanalysissimransandhu

THE KEY FINDINGS

Finding 1: The Data Problem (Most Important Finding)

135,080 rows, 24.9% of the entire dataset, had no CustomerID. This was the most operationally important finding in the project. Without catching and filtering those rows first, every customer segmentation figure, every retention calculation, and every ranked output in this analysis would have been wrong. Good analysis is not just about building reports. It is about knowing when your inputs are unreliable before anyone downstream acts on them. This is what data governance looks like in practice.

Finding 2: The Retention Opportunity

45% of customers bought once and never came back. Loyal customers, just 23% of the base, spend 16 times more than one-time buyers.

If 10% of one-time buyers made a second purchase at the occasional buyer average, that is an additional £82,000 in revenue from a single retention initiative. The data does not tell you why they did not come back. But it tells you clearly that converting even a small fraction of them is worth more than acquiring new customers.

Finding 3: The Seasonal Pattern

February revenue dropped £167,732 in a single month. That is the sharpest single month fall in the entire dataset. This is not random. Combined with the average order value of £1,394 per order, this business is clearly driven by B2B wholesale buyers purchasing in seasonal cycles, not steady consumer demand.

Finding 4: Cost Price Efficiency (Phase 2 Extension)

Three additional queries looked at which products generate the least revenue per unit handled, how the full product range can be split by commercial priority, and how unit prices moved month on month. The results show clear patterns: some products move thousands of units for pennies per unit in revenue, and certain products show compounding price increases across consecutive months. Both are the kind of findings that would trigger a commercial conversation in a buying environment.

SQL ANALYSIS: ALL QUERIES

Query 1. Which products generate the highest revenue and how concentrated is that revenue?

The first thing any retail business wants to know is what is making them money. I grouped all transactions by product description, summed up the revenue for each, and sorted from highest to lowest.

```
SELECT  
  Description AS ProductName,
```

```

ROUND(SUM(Quantity * UnitPrice), 2) AS TotalRevenue,
SUM(Quantity) AS TotalQuantitySold
FROM "Online Retail"
WHERE Quantity > 0 AND UnitPrice > 0 AND Description IS NOT NULL
GROUP BY Description
ORDER BY TotalRevenue DESC
LIMIT 10;

```

Query 1b. What percentage of total revenue do the top 10 products represent?

```

SELECT
ROUND(SUM(TopRevenue), 2) AS Top10ProductRevenue,
(SELECT ROUND(SUM(Quantity * UnitPrice), 2)
FROM "Online Retail"
WHERE Quantity > 0 AND UnitPrice > 0) AS TotalRevenue,
ROUND(100.0 * SUM(TopRevenue) /
(SELECT SUM(Quantity * UnitPrice)
FROM "Online Retail"
WHERE Quantity > 0 AND UnitPrice > 0), 2) AS PercentageOfTotal
FROM (
SELECT SUM(Quantity * UnitPrice) AS TopRevenue
FROM "Online Retail"
WHERE Quantity > 0 AND UnitPrice > 0 AND Description IS NOT NULL
GROUP BY Description
ORDER BY TopRevenue DESC
LIMIT 10
);

```

RESULT: The top 10 products generate £1.15M, representing 10.9% of total revenue (£10.6M), with DOTCOM POSTAGE leading this group. This is lower than the typical Pareto distribution, which suggests no single product dominates and inventory risk is spread across the range.

Query 2. How geographically concentrated is revenue and where is the strategic risk?

```

SELECT
Country,
ROUND(SUM(Quantity * UnitPrice), 2) AS TotalRevenue,
COUNT(DISTINCT InvoiceNo) AS NumberOfOrders
FROM "Online Retail"
WHERE Quantity > 0 AND UnitPrice > 0 AND CustomerID IS NOT NULL
GROUP BY Country
ORDER BY TotalRevenue DESC
LIMIT 10;

```

RESULT: The UK accounts for £9,025,222, which is 85% of total revenue. Netherlands (£285K), Ireland (£263K) and Germany (£221K) are the next largest markets but all significantly smaller. This level of UK concentration is a commercial risk if domestic demand softens.

Query 3. What is the average order value for top customers?

Knowing your top customers matters because losing one of them has an outsized impact. I pulled the top 15 customers by total spend and calculated their average order value.

```

SELECT
CustomerID,
ROUND(SUM(Quantity * UnitPrice), 2) AS TotalSpent,
COUNT(DISTINCT InvoiceNo) AS NumberOfOrders,
ROUND(SUM(Quantity * UnitPrice) / COUNT(DISTINCT InvoiceNo), 2) AS AvgOrderValue
FROM "Online Retail"
WHERE Quantity > 0 AND UnitPrice > 0 AND CustomerID IS NOT NULL
GROUP BY CustomerID
ORDER BY TotalSpent DESC
LIMIT 15;

```

RESULT: The top customer spent £280,000 across 201 orders, averaging £1,394 per order. These are wholesale B2B buyers, not individual consumers. Revenue is heavily concentrated among a small number of accounts, which makes retaining them the most commercially important activity in the business.

Query 4. How is revenue changing month by month and is the business growing?

I used a CTE to first calculate monthly revenue, then used the LAG window function to compare each month against the previous one. LAG lets you look back at the previous row without needing a self-join.

```
WITH MonthlyRevenue AS (  
    SELECT  
        SUBSTR(InvoiceDate, 7, 4) AS Year,  
        SUBSTR(InvoiceDate, 4, 2) AS Month,  
        ROUND(SUM(Quantity * UnitPrice), 2) AS Revenue  
    FROM "Online Retail"  
    WHERE Quantity > 0 AND UnitPrice > 0  
    GROUP BY Year, Month  
) SELECT  
    Year,  
    Month,  
    Revenue,  
    LAG(Revenue) OVER (ORDER BY Year, Month) AS PreviousMonthRevenue,  
    ROUND(Revenue - LAG(Revenue) OVER (ORDER BY Year, Month), 2) AS MonthlyChange  
FROM MonthlyRevenue  
ORDER BY Year, Month;
```

RESULT: Revenue started at £823,746 in December 2010 and peaked in November 2011. The trend was not linear. February dropped £167,732 before recovering in March. This pattern points to a B2B seasonal procurement cycle where bulk orders drive sharp month-on-month swings rather than steady consumer demand. The LAG function made it straightforward to spot these fluctuations without manual calculation.

Query 5. What are the low revenue products to consider for discontinuation?

The flip side of knowing your best sellers is knowing your worst. Products that barely generate any revenue are consuming storage and management attention. I pulled the bottom performers to flag them for review.

```
SELECT  
    Description AS ProductName,  
    ROUND(SUM(Quantity * UnitPrice), 2) AS TotalRevenue,  
    COUNT(DISTINCT InvoiceNo) AS NumberOfSales,  
    SUM(Quantity) AS TotalQuantitySold  
FROM "Online Retail"  
WHERE Quantity > 0 AND UnitPrice > 0 AND Description IS NOT NULL  
GROUP BY Description  
ORDER BY TotalRevenue ASC  
LIMIT 15;
```

RESULT: Several products generated under £5 in total revenue over the entire 12-month period. These items show minimal sales volume and limited market demand. They consume inventory resource with no commercial return and are candidates for a ranging review.

Query 6. Customer segmentation by buying pattern

Not all customers are equal. I used a CASE WHEN statement to segment customers into three groups based on how many times they purchased, then compared average spend across groups.

```
SELECT  
    CASE  
        WHEN PurchaseCount = 1 THEN 'One-time Buyer'  
        WHEN PurchaseCount BETWEEN 2 AND 5 THEN 'Occasional Buyer'  
        ELSE 'Loyal Customer'  
    END AS CustomerType,  
    COUNT(CustomerID) AS NumberOfCustomers,  
    ROUND(AVG(TotalSpent), 2) AS AvgSpendingPerCustomer  
FROM (  
    SELECT  
        CustomerID,  
        COUNT(DISTINCT InvoiceNo) AS PurchaseCount,  
        SUM(Quantity * UnitPrice) AS TotalSpent  
    FROM "Online Retail"  
    WHERE Quantity > 0 AND UnitPrice > 0 AND CustomerID IS NOT NULL  
    GROUP BY CustomerID  
)
```

```
GROUP BY CustomerType;
```

RESULT: 45% of customers are one-time buyers, spending £413 on average. 32% are occasional buyers, averaging £1,211. Loyal customers, just 23% of the base, spend £6,773 each. That gap is the commercial case for retention over acquisition.

Query 7. Who are the top customers ranked by total revenue?

I used the RANK() window function to assign a rank to each customer based on total revenue. Unlike GROUP BY which collapses rows, window functions let you add a calculated column while keeping the full row intact.

```
SELECT
  CustomerID,
  ROUND(SUM(Quantity * UnitPrice), 2) AS TotalRevenue,
  RANK() OVER (ORDER BY SUM(Quantity * UnitPrice) DESC) AS RevenueRank
FROM "Online Retail"
WHERE Quantity > 0 AND UnitPrice > 0 AND CustomerID IS NOT NULL
GROUP BY CustomerID
LIMIT 15;
```

RESULT: Customer 14646 ranked first with £280,206, followed by Customer 18102 at £259,657. By rank 3 revenue drops to £194,550 and falls sharply from there. A small number of accounts drive a disproportionate share of revenue.

Query 8. Which one-time buyers contribute most to country revenue and what is the retention risk?

This query required joining two separate aggregations using a CTE JOIN. Both sides of the join are derived summaries rather than raw tables, which makes this a more advanced technique than a standard join.

```
WITH CustomerSummary AS (
  SELECT
    CustomerID,
    Country,
    COUNT(DISTINCT InvoiceNo) AS TotalOrders,
    ROUND(SUM(Quantity * UnitPrice), 2) AS CustomerRevenue
  FROM "Online Retail"
  WHERE Quantity > 0 AND UnitPrice > 0 AND CustomerID IS NOT
  NULL GROUP BY CustomerID, Country),
CountrySummary AS (
  SELECT
    Country,
    ROUND(SUM(Quantity * UnitPrice), 2) AS CountryRevenue,
    COUNT(DISTINCT CustomerID) AS TotalCustomers
  FROM "Online Retail"
  WHERE Quantity > 0 AND UnitPrice > 0 AND CustomerID IS NOT NULL
  GROUP BY Country)
SELECT
  cs.CustomerID,
  cs.Country,
  cs.TotalOrders,
  cs.CustomerRevenue,
  ct.CountryRevenue,
  ct.TotalCustomers,
  ROUND(100.0 * cs.CustomerRevenue / ct.CountryRevenue, 2) AS CustomerShareOfCountryRevenue
FROM CustomerSummary cs
JOIN CountrySummary ct ON cs.Country = ct.Country
WHERE cs.TotalOrders = 1
ORDER BY cs.CustomerRevenue DESC
LIMIT 20;
```

RESULT: The top 20 one-time buyers are all UK-based, spending between £1,882 and £2,062 in a single visit. Converting even a small fraction of these accounts to repeat buyers would have a meaningful revenue impact. This query shows how a JOIN across two aggregated CTEs can surface actionable insight that neither table could produce alone.

Query 9. Data quality and governance validation

This query ran before anything else. Every finding in the analysis depends on knowing the data is clean first.

```
SELECT
  COUNT(*) AS TotalRows,
  COUNT(CustomerID) AS RowsWithCustomerID,
  COUNT(*) - COUNT(CustomerID) AS MissingCustomerIDs,
  SUM(CASE WHEN Quantity <= 0 THEN 1 ELSE 0 END) AS NegativeQuantityRows,
  SUM(CASE WHEN UnitPrice <= 0 THEN 1 ELSE 0 END) AS ZeroPriceRows,
  ROUND(100.0 * COUNT(CustomerID) / COUNT(*), 2) AS DataCompletenessRate FROM "Online Retail";
```

RESULT: 541,909 total rows. 135,080 rows, 24.9% of the dataset, had no CustomerID. Without filtering these first, every customer metric in the analysis would have been wrong. Data governance comes before analysis. Always.

PHASE 2: COST PRICE EFFICIENCY ANALYSIS

After completing the core revenue and customer analysis I built three more queries to look at cost price efficiency. The question I was trying to answer was a practical one: which products are moving high volume but generating almost nothing per unit, and are there any patterns in how prices have moved over time that would flag a commercial problem?

These queries were not designed to replicate a full commercial cost model. They were built to show the same analytical thinking a buying team uses when reviewing supplier cost price performance.

Query 10a. Revenue per unit by product

Products that move a lot of units but generate very little revenue per unit are worth flagging. The operational cost of handling 8,000 units of something that makes £0.04 each is not justified by the commercial return. I filtered for products that sold more than 100 units to focus on meaningful volumes.

```
SELECT
  Description AS ProductName,
  ROUND(SUM(Quantity * UnitPrice), 2) AS TotalRevenue,
  SUM(Quantity) AS TotalUnitsSold,
  ROUND(AVG(UnitPrice), 2) AS AvgUnitPrice,
  ROUND(SUM(Quantity * UnitPrice) / SUM(Quantity), 2) AS RevenuePerUnit
FROM "Online Retail"
WHERE Quantity > 0 AND UnitPrice > 0 AND Description IS NOT NULL
GROUP BY Description
HAVING SUM(Quantity) > 100
ORDER BY RevenuePerUnit ASC
LIMIT 15;
```

RESULT: POPART WOODEN PENCILS ASST moved 8,900 units at £0.04 revenue per unit. CARTOON PENCIL SHARPENERS moved 3,821 units at £0.07. ESSENTIAL BALM moved 5,864 units at £0.07. LETTER SHAPE PENCIL SHARPENER moved 3,333 units at £0.07. These are the products where operational cost is likely to exceed the commercial return per unit. In a buying context they are the first conversation to have with the supplier.

Query 10b. Product range commercial action categories

I wanted to go further than just listing low margin products. I built a CTE to calculate each product's revenue share across the whole dataset, then used CASE WHEN to put every product into a commercial action category based on its revenue share and revenue per unit. The idea is to give a buying team a prioritised list rather than just a ranked table.

```

WITH ProductMetrics AS (
  SELECT
    Description AS ProductName,
    SUM(Quantity) AS TotalVolume,
    ROUND(SUM(Quantity * UnitPrice), 2) AS TotalRevenue,
    ROUND(SUM(Quantity * UnitPrice) / SUM(Quantity), 2) AS RevenuePerUnit,
    ROUND(100.0 * SUM(Quantity * UnitPrice) /
      SUM(SUM(Quantity * UnitPrice)) OVER (), 2) AS RevenueShare
  FROM "Online Retail"
  WHERE Quantity > 0 AND UnitPrice > 0 AND Description IS NOT NULL
  GROUP BY Description
) SELECT
  ProductName,
  TotalVolume,
  TotalRevenue,
  RevenuePerUnit,
  RevenueShare,
  CASE
    WHEN RevenueShare > 1.0 AND RevenuePerUnit < 1.50 THEN 'Priority Renegotiation'
    WHEN RevenueShare > 0.5 AND RevenuePerUnit < 1.00 THEN 'Cost Review Needed'
    WHEN TotalVolume > 5000 AND RevenuePerUnit < 0.80 THEN 'Volume Discount Opportunity'
    ELSE 'Monitor'
  END AS CommercialAction
FROM ProductMetrics
ORDER BY TotalRevenue DESC
LIMIT 20;

```

RESULT: PAPER CRAFT, LITTLE BIRDIE moved 80,995 units at £2.08 per unit, representing 1.58% of total revenue. MEDIUM CERAMIC TOP STORAGE JAR moved 78,033 units at £1.06 per unit. These products sit at high volume with low unit revenue and would be Priority Renegotiation or Cost Review candidates. The CASE WHEN logic means a buyer can filter by action category rather than having to make that judgment from a raw ranked list.

Query 10c. Month on month price movement

The last query used LAG partitioned by product to compare each product's average monthly price against the prior month. The goal was to find products where price moved sharply enough in either direction to flag as either a cost inflation risk or a deflationary opportunity. This is the same logic used to monitor supplier pricing over time.

```

WITH MonthlyProductRevenue AS (
  SELECT
    SUBSTR(InvoiceDate, 7, 4) AS Year,
    SUBSTR(InvoiceDate, 4, 2) AS Month,
    Description AS ProductName,
    ROUND(AVG(UnitPrice), 2) AS AvgMonthlyPrice
  FROM "Online Retail"
  WHERE Quantity > 0 AND UnitPrice > 0 AND Description IS NOT
  NULL GROUP BY Year, Month, Description),
PriceMovement AS (
  SELECT
    ProductName,
    Month,
    AvgMonthlyPrice,
    LAG(AvgMonthlyPrice) OVER (PARTITION BY ProductName ORDER BY Year, Month) AS
  PrevMonthPrice,
    ROUND(AvgMonthlyPrice - LAG(AvgMonthlyPrice)
    OVER (PARTITION BY ProductName ORDER BY Year, Month), 2) AS PriceChange
  FROM MonthlyProductRevenue
) SELECT
  ProductName,
  Month,
  AvgMonthlyPrice,
  PrevMonthPrice,
  PriceChange,
  CASE
    WHEN PriceChange > 0.60 THEN 'Cost Inflation Risk'
    WHEN PriceChange < -0.60 THEN 'Deflationary Opportunity'
    ELSE 'Stable'
  END AS PriceFlag
FROM PriceMovement
WHERE PriceChange IS NOT NULL AND ABS(PriceChange) > 0.60
ORDER BY ABS(PriceChange) DESC
LIMIT 20;

```

RESULT: DOTCOM POSTAGE increased from £417 in month 10 to £785 in month 11 and £1,242 in month 12. That is two consecutive months of compounding price increases. In a managed buying environment this would trigger an immediate review. AMAZON FEE dropped £13,321 in a single month, which points to a contract change rather than organic price movement. The LAG partitioned by product is what makes this possible without writing a separate query for each product.

TABLEAU DASHBOARD

Three views are published on Tableau Public. The SQL findings translated into visual outputs.

View 1: Monthly Revenue Trend. Line chart showing the month-on-month movement from Query 4, including the February drop and November peak.

View 2: Revenue Per Unit by Product. Bar chart showing the lowest margin products by volume from Query 10a, ordered by revenue per unit ascending.

View 3: Month on Month Price Movement. Chart showing price change by product from Query 10c, flagging cost inflation and deflationary movements.

Live dashboard:

CONCLUSION

This dataset covers a UK-based wholesale retailer with over half a million transactions. The analysis shows the business is heavily reliant on a small number of high-value B2B accounts, concentrated almost entirely in the UK, with predictable seasonal dips that the business does not appear to plan around.

The Phase 2 cost price work found products moving thousands of units at pennies per unit in revenue, and price movements compounding across consecutive months without any obvious corrective action. None of this required new data. It required asking the right questions of the data that already existed.

The opportunity is straightforward: deepen relationships with existing high-value accounts before chasing new ones, use the seasonal pattern to plan rather than react, review the product range using the cost price efficiency data, and build a price monitoring process so inflation spikes are caught early rather than after they have compounded.